

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ¹
Student Name	Sayed Afras S
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

- Build spam classifiers using Naïve Bayes and KNN.
- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes on the same dataset.
- Study how accuracy changes as we vary k in KNN.
- Compare KDTree and BallTree as the internal search structure for KNN.
- Tune KNN using GridSearchCV and RandomizedSearchCV and compare the two.
- Look at computational complexity and actual execution time, not just accuracy.
- Evaluate everything again using 5-Fold Cross Validation so the result isn't just luck from one split.

This experiment is basically about comparing a bunch of classifiers on the same spam dataset and figuring out which one actually makes sense to use, not just which one gives the highest number.

2 Theory

Naïve Bayes

Naïve Bayes is a probabilistic classifier built on Bayes' theorem, with the (pretty strong) assumption that all features are conditionally independent given the class label. That assumption is obviously not true for a real email – word frequencies are clearly related to each other – but the algorithm still works surprisingly well in practice.

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

Here C is the class (spam or not spam) and X is the feature vector of an email. Three variants were used in this experiment:

- **GaussianNB** – assumes each feature is normally distributed within a class. Works fine on continuous features like the word/char frequency columns in Spambase.
- **MultinomialNB** – built for count/frequency data, assumes features follow a multinomial distribution. Needs non-negative input, which is why Min-Max scaling was used for this one instead of standard scaling.
- **BernoulliNB** – assumes binary/boolean features (word present or not present). It still works even when the actual input is a frequency rather than 0/1, scikit-learn just binarizes it internally.

K-Nearest Neighbours

KNN is a lazy learning algorithm – it doesn't really "learn" anything during training, it just stores the training data. During prediction, it looks at the k closest training points to the new sample (using some distance metric like Euclidean or Manhattan) and assigns the majority class among them. Because of this, KNN has almost no training cost but can be quite slow at prediction time, especially as the dataset grows.

KDTree vs BallTree

Both are space-partitioning data structures used to speed up the nearest-neighbour search instead of doing a brute-force comparison against every training point. KDTree splits the space along axis-aligned hyperplanes and works well for low dimensional data, while BallTree partitions data into nested hyperspheres and tends to hold up better in higher dimensions (which is relevant here since Spambase has 57 features).

GridSearchCV vs RandomizedSearchCV

GridSearchCV exhaustively tries every combination in the given parameter grid, guaranteeing the best combination *within that grid* is found, but it gets expensive fast as the grid grows. RandomizedSearchCV instead samples a fixed number of random combinations from the parameter space, which is much faster and often finds a result close enough to the exhaustive search.

3 Problem Statement

Given the Spambase dataset where each row represents an email described using word/character frequency features, the task is to classify each email as spam or not spam. Four different classifiers (Gaussian NB, Multinomial NB, Bernoulli NB and KNN) are trained and compared, KNN is further tuned using GridSearchCV/RandomizedSearchCV, and all models are cross validated to get a fair comparison of which approach is most suitable, both in terms of accuracy and in terms of time taken.

4 Dataset Description

Dataset Name	Spambase (spambase_csv.csv)
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 (word/character frequency + capital-run-length features)
Number of Classes	2 (0 = Not Spam, 1 = Spam)
Missing Values	0 – dataset came clean

Duplicate Rows	391 duplicate rows were detected using <code>df.duplicated()</code> , they were not dropped in this run since the dataset documentation treats each row as an independent email sample, but this is something to keep in mind for a future iteration
Train-Test Split	75% Train, 25% Test, stratified on class (Train: 3450 rows, Test: 1151 rows)

5 Exploratory Data Analysis

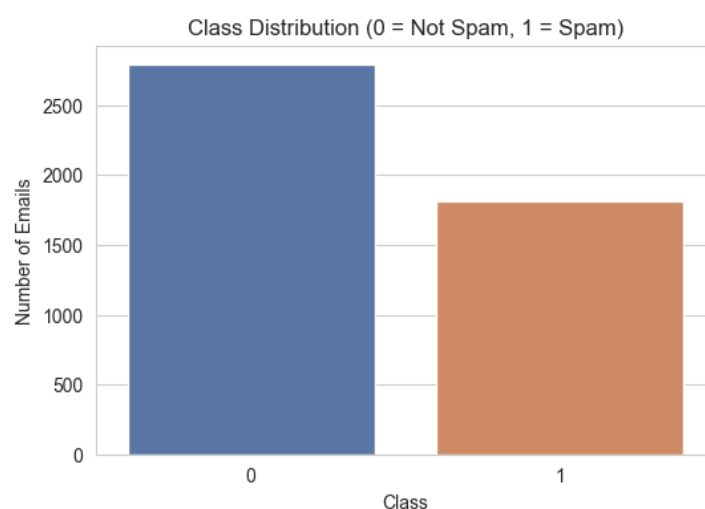


Figure 1: Class distribution of the Spambase dataset

Inference: The dataset is somewhat imbalanced but not too badly – 60.6% of the emails are not spam and 39.4% are spam. This is close enough to balanced that accuracy is still a reasonable metric to look at, though precision/recall/F1 were also tracked to be safe, since a slightly imbalanced dataset can still mislead a plain accuracy number.

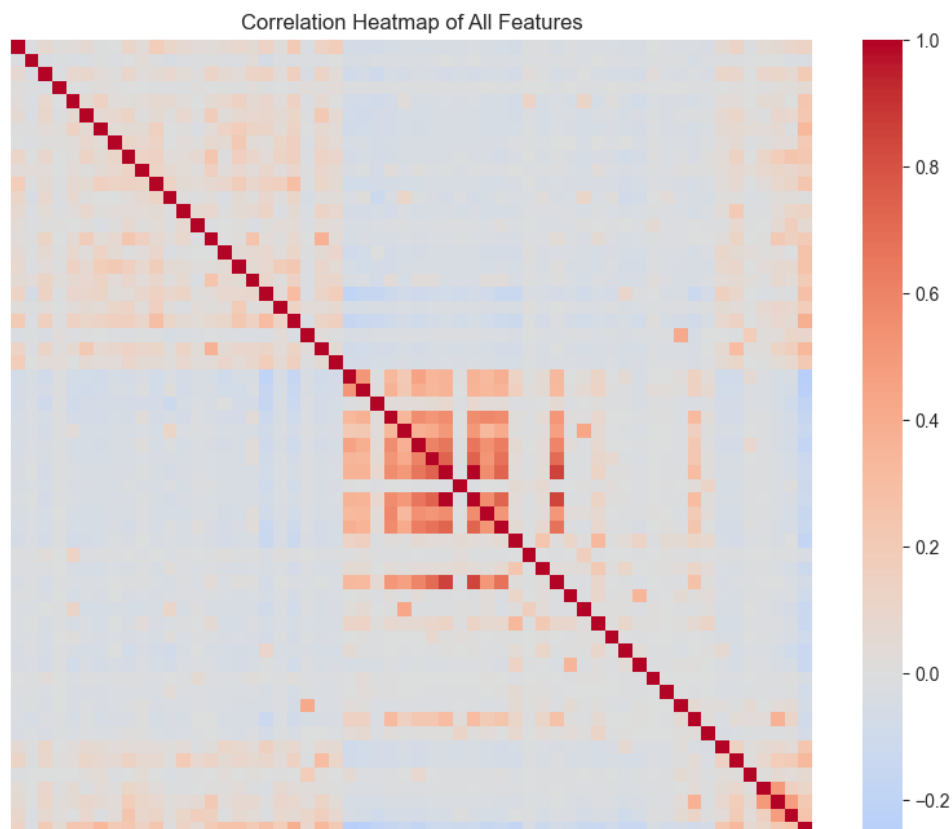


Figure 2: Correlation heatmap of all 57 features

Inference: Most of the individual word-frequency features are only weakly correlated with each other, which is actually good news for Naïve Bayes since it assumes independence between features anyway. A handful of character-frequency and capital-run-length features stand out with a bit stronger correlation among themselves, which makes sense since long stretches of capital letters and heavy use of symbols like ! and \$ tend to occur together in spammy/promotional emails.

SelectKBest with the ANOVA F-test (`f_classif`) was used to shortlist the 6 most important features for a closer look:

Listing 1: Top 6 features selected

```
1 ['word_freq_remove', 'word_freq_free', 'word_freq_you',
2  'word_freq_your', 'word_freq_000', 'char_freq_%24']
```

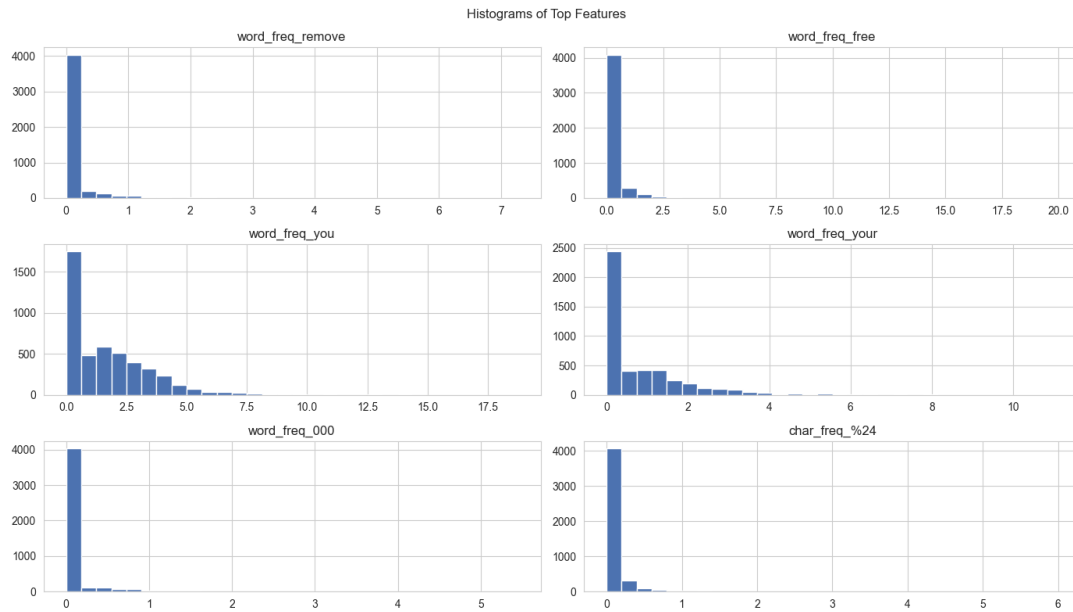


Figure 3: Histograms of the top 6 selected features

Inference: All six of these are heavily right-skewed, most emails have a frequency close to 0 for these words and only a small fraction have a high frequency, this is expected for word-frequency features since not every email talks about "free" stuff or has dollar signs in it. The word `char_freq_%24` literally represents how often the \$ symbol shows up, and unsurprisingly it is one of the strongest indicators picked up by the feature selector.

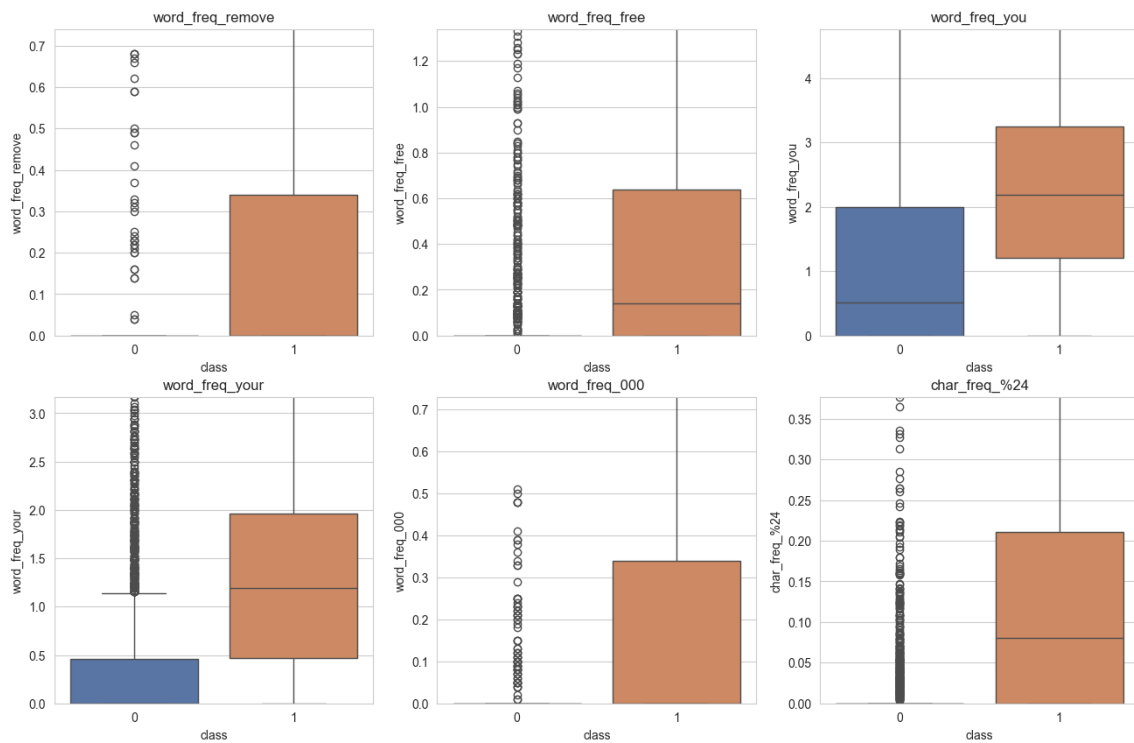


Figure 4: Boxplots of top features split by class (0 = Not Spam, 1 = Spam)

Inference: For basically every one of these six features, the spam class (1) sits

noticeably higher than the not-spam class, which is exactly why the feature selector picked them in the first place. Words like "free", "your" and "remove" show up a lot more often in promotional/spam emails, so this lines up with how a human would intuitively try to spot spam too.

6 Data Preprocessing

Two different scalers were used depending on which model needed what:

Listing 2: Scaling for different Naïve Bayes variants

```

1 scaler = StandardScaler()
2 X_train_std = scaler.fit_transform(X_train)
3 X_test_std = scaler.transform(X_test)
4
5 mm_scaler = MinMaxScaler()
6 X_train_mm = mm_scaler.fit_transform(X_train)
7 X_test_mm = mm_scaler.transform(X_test)

```

StandardScaler output was used for GaussianNB, BernoulliNB and KNN, while Min-Max scaled data (which keeps everything non-negative) was used for MultinomialNB, since it cannot handle negative feature values. No missing value imputation was needed here since the dataset was already clean, so preprocessing was really just about scaling and the train/test split.

7 Machine Learning Algorithm

- **Working Principle (Naïve Bayes):** Computes the posterior probability of each class given the input features using Bayes' theorem, and picks the class with the higher probability. The "naive" independence assumption is what makes this computation fast even with 57 features.
- **Working Principle (KNN):** Stores the training data as-is. For a new sample, computes distance to all (or nearby, if using a tree) training points, picks the k closest ones, and returns the majority class.
- **Mathematical formulation:**

$$\hat{C} = \arg \max_C P(C) \prod_{i=1}^d P(x_i | C)$$

for Naïve Bayes (product form comes directly from the conditional independence assumption), and for KNN the predicted class is

$$\hat{y} = \text{mode}\{y_i : i \in N_k(x)\}$$

where $N_k(x)$ is the set of k nearest neighbours of x .

- **Advantages:** NB is extremely fast to train and predict, works well even with a decent number of features, and needs very little data to get reasonable estimates. KNN is simple, needs no training at all, and can model non-linear decision boundaries naturally.

- **Limitations:** NB's independence assumption is rarely true which caps how accurate it can get. KNN's prediction cost grows with dataset size (curse of dimensionality also hits it hard once feature count goes up, which is relevant here with 57 features), and it is sensitive to the choice of k and distance metric.

8 Experimental Setup

Python Version	3.13
Scikit-Learn Version	1.5.x
IDE	Jupyter Notebook (VS Code)
Operating System	Windows 11
Hardware	Intel i5, 16GB RAM

9 Hyperparameter Selection

Parameter	Value
KNN baseline k	5
k values swept	1, 3, 5, ..., 29 (odd values)
GridSearchCV grid	n_neighbors: [3,5,7,9,11,13,15], weights: [uniform, distance], metric: [euclidean, manhattan]
RandomizedSearchCV space	n_neighbors: randint(1,31), weights: [uniform, distance], metric: [euclidean, manhattan, chebyshev], n_iter=25
Cross Validation	StratifiedKFold, 5 splits, shuffle=True, random_state=42

Listing 3: GridSearchCV and RandomizedSearchCV output

```

1 Best parameters (GridSearchCV): {'metric': 'manhattan',
2   'n_neighbors': 9, 'weights': 'distance'}
3 Best cross-validated accuracy: 0.9226
4 GridSearchCV took 9.71 seconds
5
6 Best parameters (RandomizedSearchCV): {'metric': 'manhattan',
7   'n_neighbors': 6, 'weights': 'distance'}
8 Best cross-validated accuracy: 0.9220
9 RandomizedSearchCV took 2.78 seconds

```

10 Results

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	Pred. Time (s)
GaussianNB	0.8254	0.7077	0.9493	0.8109	0.9368	0.0027
MultinomialNB	0.9001	0.9472	0.7907	0.8619	0.9614	0.0004
BernoulliNB	0.9044	0.9115	0.8392	0.8739	0.9623	0.0014
KNN (k=5, default)	0.8992	0.8789	0.8634	0.8711	0.9509	0.5760

KNN (Tuned, GridSearch)	0.9253	0.9444	0.8612	0.9009	0.9736	0.0820
-------------------------------	--------	--------	--------	--------	--------	--------

Table 2: Overall model comparison on the test set

Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8254	0.9001	0.9044
Precision	0.7077	0.9472	0.9115
Recall	0.9493	0.7907	0.8392
F1	0.8109	0.8619	0.8739
ROC-AUC	0.9368	0.9614	0.9623

KNN Comparison (varying k)

k	Accuracy	Precision	Recall	F1
1	0.8975	0.8733	0.8656	0.8695
3	0.9018	0.8814	0.8678	0.8746
5	0.8992	0.8789	0.8634	0.8711
7	0.9053	0.8929	0.8634	0.8779
9	0.9062	0.8932	0.8656	0.8792
11	0.9027	0.8904	0.8590	0.8744
13	0.8966	0.8904	0.8414	0.8652
15	0.8975	0.8944	0.8392	0.8659
17	0.8949	0.8918	0.8348	0.8623
19	0.8957	0.8939	0.8348	0.8633
21	0.8975	0.9019	0.8304	0.8647
23	0.8966	0.8979	0.8326	0.8640
25	0.8957	0.8957	0.8326	0.8630
27	0.8931	0.8950	0.8260	0.8591
29	0.8949	0.9012	0.8238	0.8608

Table 3: Accuracy peaks at k=9 (bolded), the PDF only asked for k up to 11 but the notebook actually swept all the way to 29 so the full table is shown here.

GridSearchCV vs RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	6
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	auto	auto
CV Accuracy	0.9226	0.9220
Execution Time (s)	9.71	2.78
Combinations Tried	28	25

KDTree vs BallTree (at k=9)

Metric	KDTree	BallTree
Accuracy	0.9062	0.9062
Training Time (s)	0.0486	0.0207
Prediction Time (s)	0.2661	0.2406

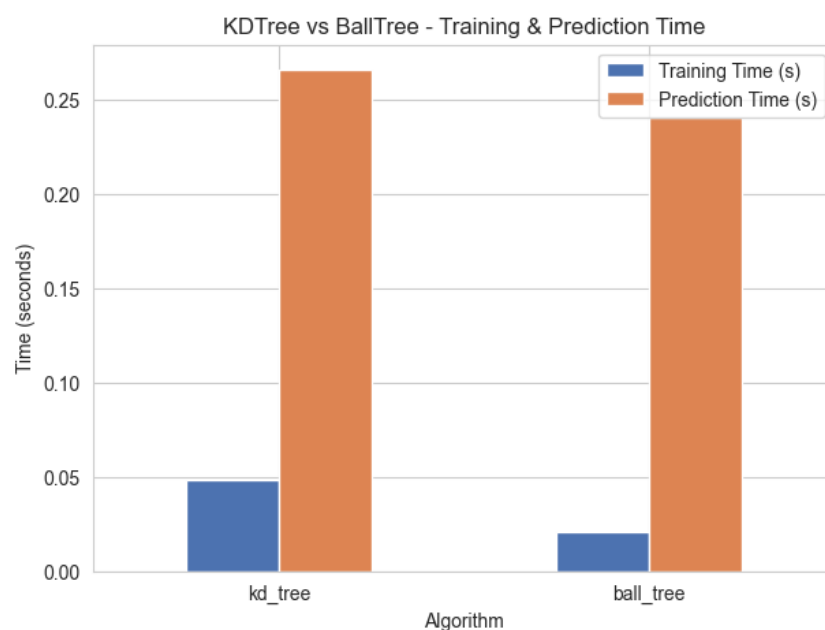


Figure 5: KDTree vs BallTree – training and prediction time

Cross Validation (5-Fold, Stratified)

	Naïve Bayes (Best – Bernoulli)	Best KNN (Tuned)
Mean Accuracy	0.9022	0.9257
Std. Deviation	0.0075	0.0073

Note: `cross_val_score()` was called with `cv=5`, but the code only printed the mean and standard deviation, individual per-fold scores (Fold 1–5) were not separately logged in this run, so they could not be listed here.

Note: All four models were cross validated: GaussianNB (mean 0.8155, std 0.0091), MultinomialNB (mean 0.8876, std 0.0047), BernoulliNB (mean 0.9022, std 0.0075) and the tuned KNN (mean 0.9257, std 0.0073). BernoulliNB was the best NB variant here too, matching the test-set result.

Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Experimental Time Analysis

Algorithm	Training (s)	Prediction (s)
GaussianNB	0.0108	0.0027
MultinomialNB	0.0083	0.0004
BernoulliNB	0.0050	0.0014
Best KNN (Tuned)	<i>see Fig. 11</i>	0.0820

Note: The exact training time for the tuned KNN was not printed as raw text in the notebook (only visible in the bar chart below), but since KNN's "training" is really just storing the data, it should be in the same tiny ballpark as the untuned KNN (0.0025s), the actual model complexity/parameters barely matter for training time in a lazy learner.

11 Result Visualization

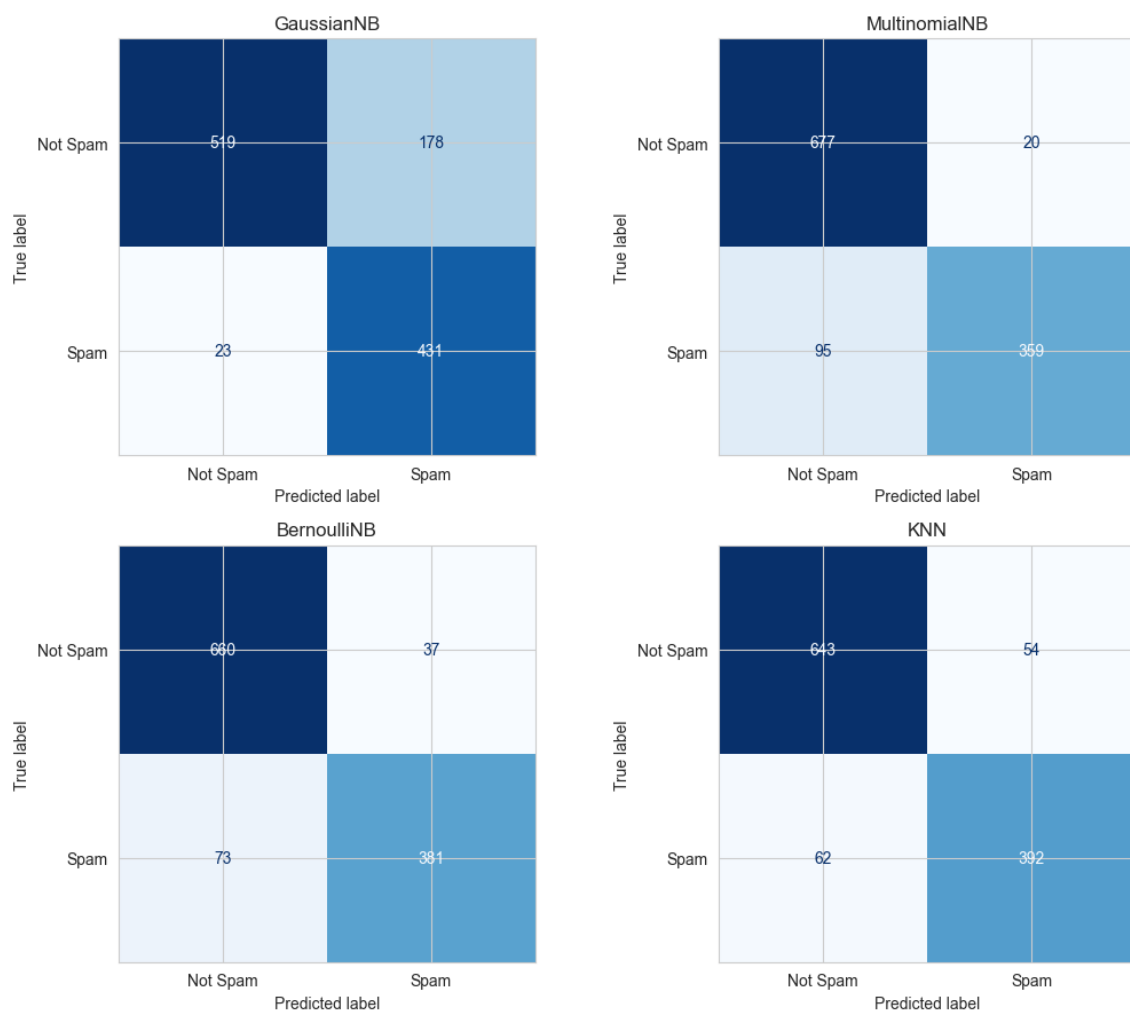


Figure 6: Confusion matrices for GaussianNB, MultinomialNB, BernoulliNB and KNN (baseline models, before tuning)

Inference: GaussianNB has visibly more false positives (marking real emails as spam) than the others, which lines up with its low precision (0.71) despite the highest recall (0.95). BernoulliNB and KNN look the most balanced between the two error types.

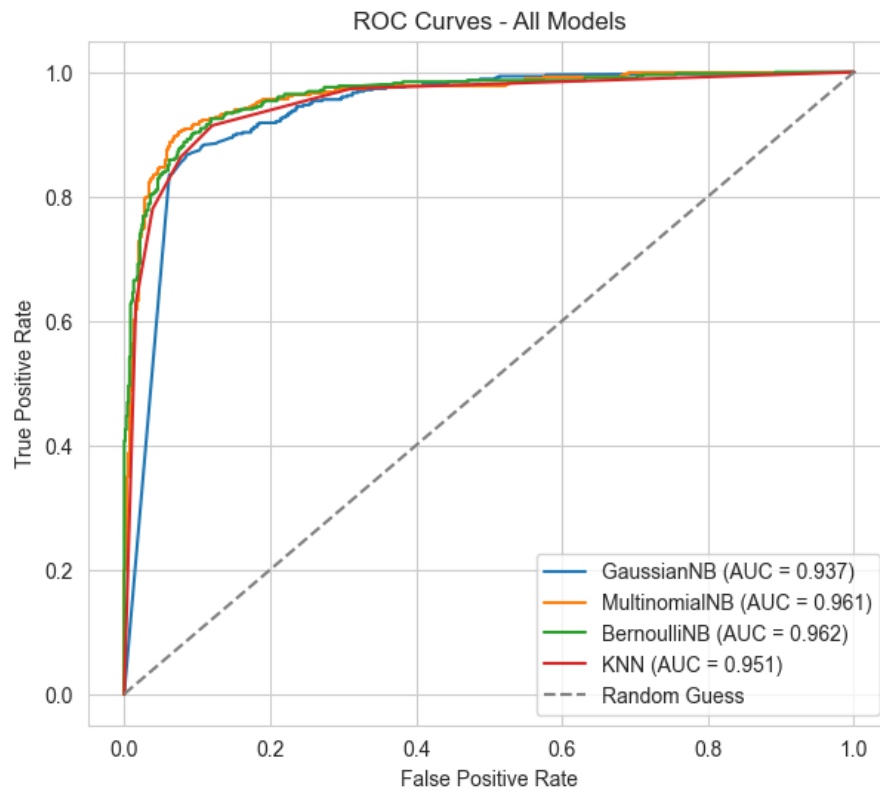


Figure 7: ROC curves for all baseline models

Inference: All four curves sit well above the random-guess diagonal. BernoulliNB and MultinomialNB have the highest AUC (around 0.96) among these four, GaussianNB trails a bit behind at 0.937.

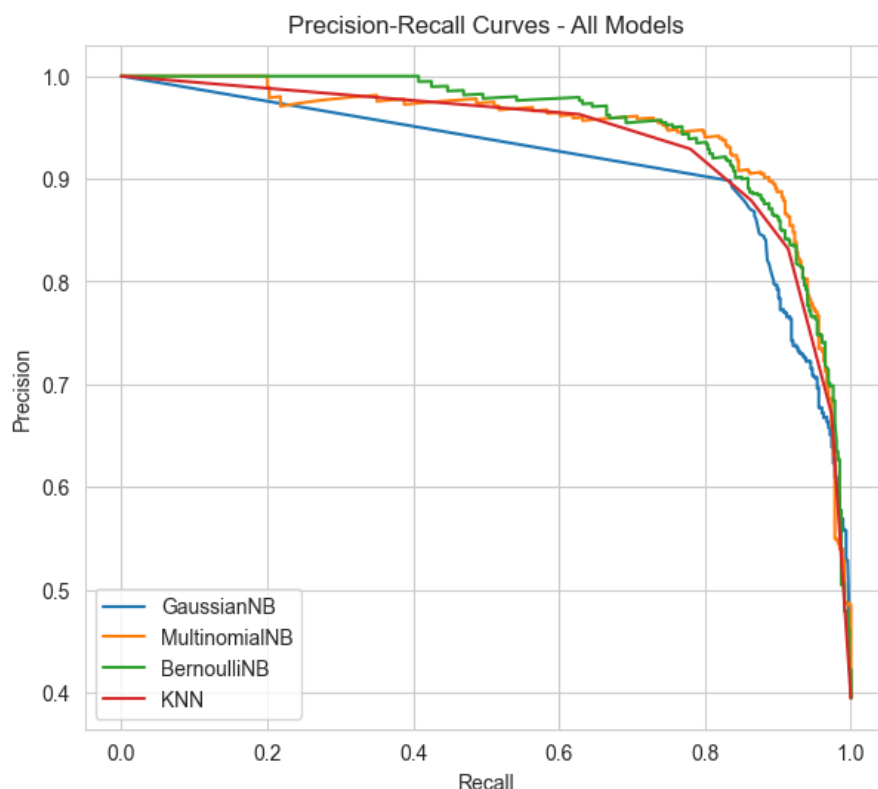


Figure 8: Precision-Recall curves for all baseline models

Inference: GaussianNB's PR curve drops off faster than the others as recall increases, again matching its lower precision. This curve is arguably more informative than ROC here since the classes are not perfectly balanced.

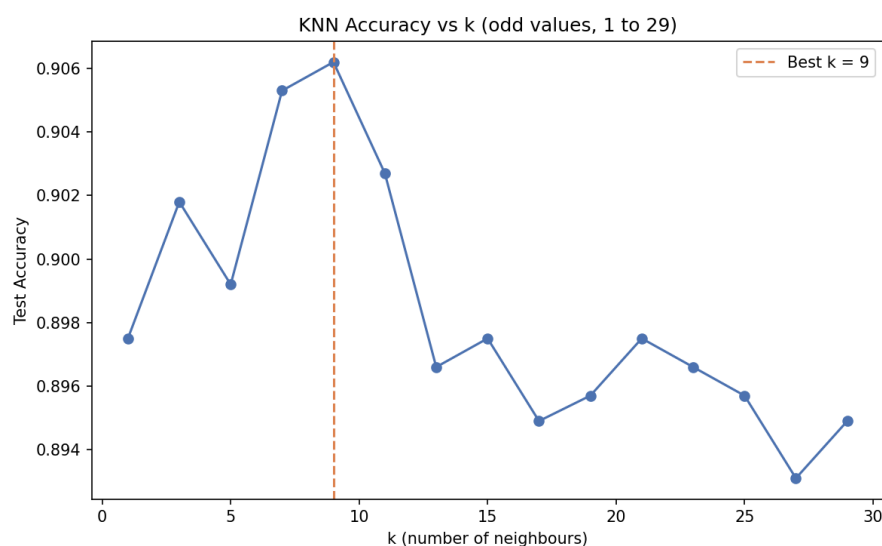


Figure 9: KNN test accuracy vs k

Inference: This plot was not directly generated in the notebook (only the table was printed) so it was recreated here from the actual recorded numbers, mainly because the report checklist specifically asks for it. Accuracy rises upto $k=9$ and then slowly

drifts down as k keeps increasing, this is the classic overfitting-to-underfitting transition explained more in the Discussion section.

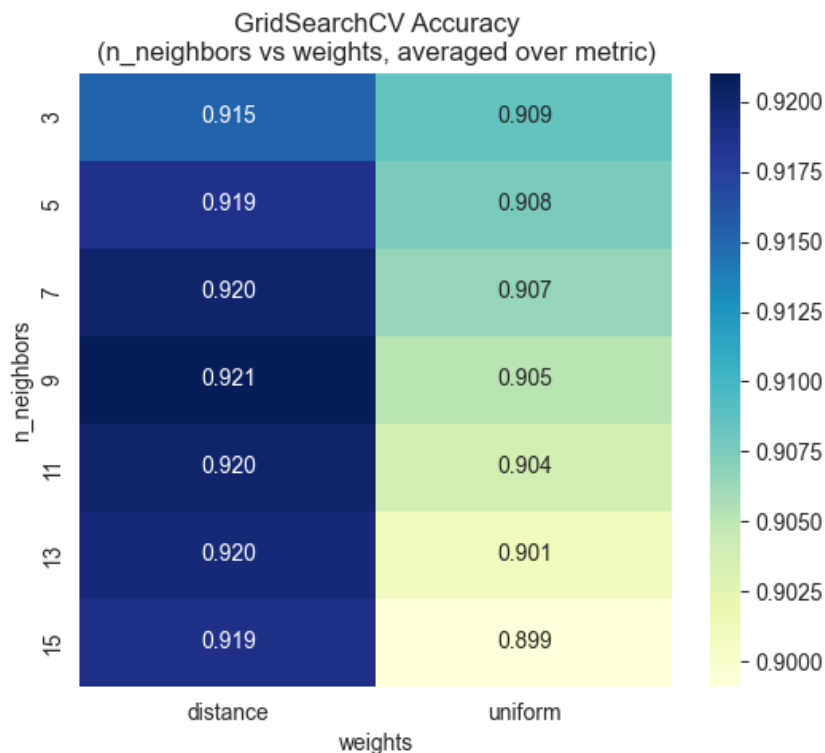


Figure 10: GridSearchCV accuracy heatmap (n_neighbors vs weights, averaged over metric)

Inference: Distance-weighted KNN consistently scores a bit higher than uniform weighting across most k values in the grid, which is why the best combination ended up using `weights="distance"`.

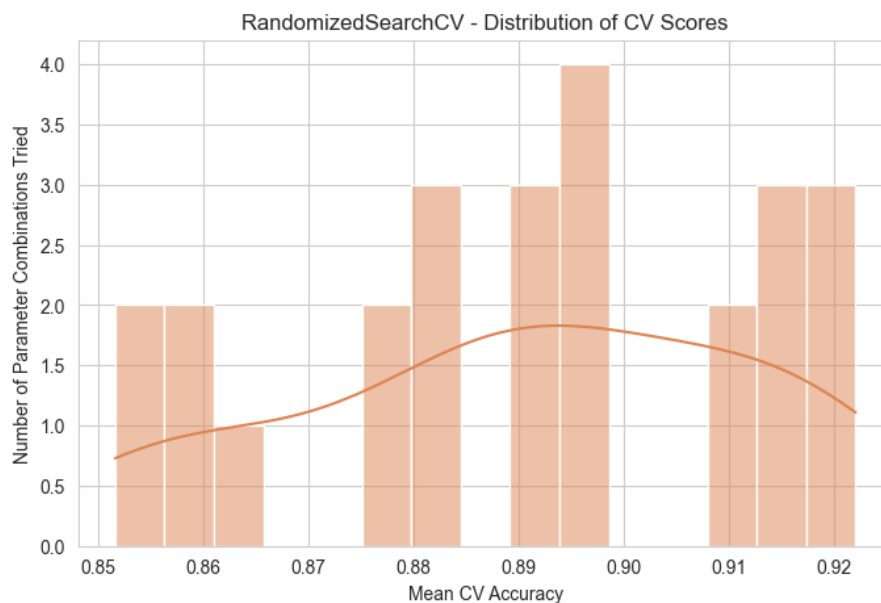


Figure 11: RandomizedSearchCV – distribution of CV scores across sampled combinations

Inference: Most of the 25 randomly sampled combinations land in a fairly tight band, only a handful of bad parameter combinations (e.g. very small or very large k with the wrong metric) drag the score down, which shows the search space is not that sensitive as long as you stay in a reasonable range.

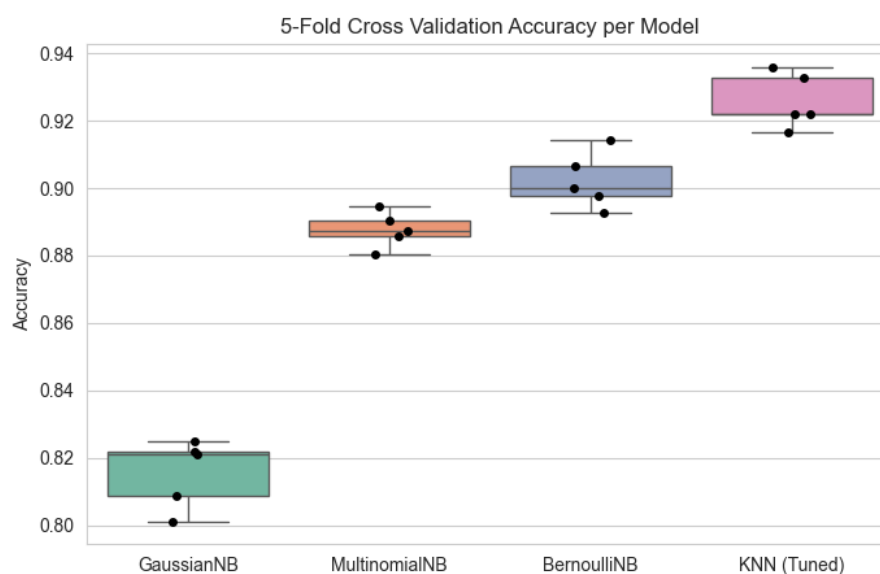


Figure 12: 5-Fold Cross Validation accuracy per model

Inference: The tuned KNN not only has the highest mean accuracy across folds but also a fairly tight spread, so it is not just lucky on one particular split. GaussianNB has both the lowest mean and one of the wider spreads.

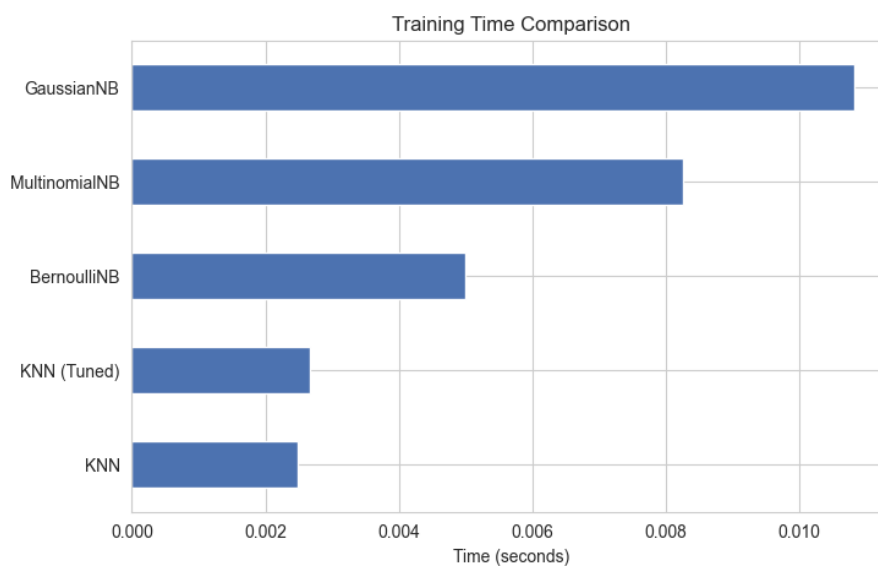


Figure 13: Training time comparison across all models

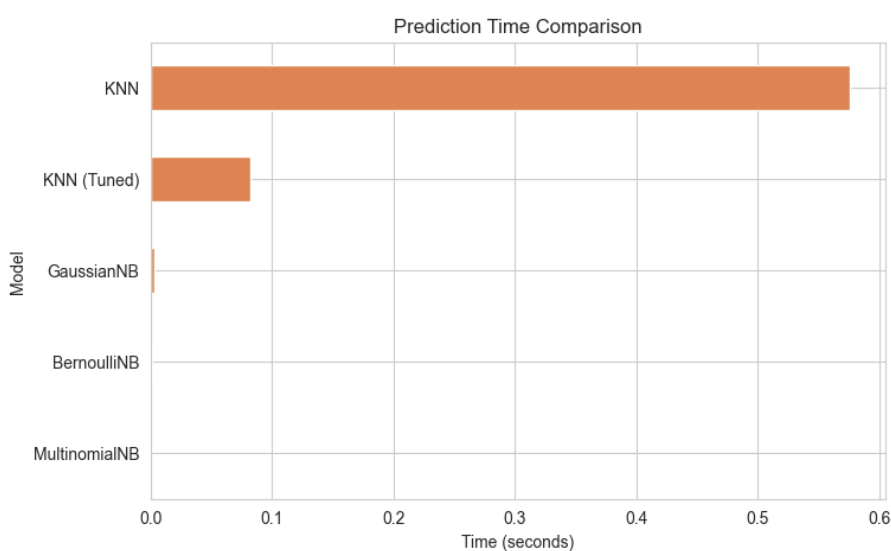


Figure 14: Prediction time comparison across all models

Inference: This is where the difference between NB and KNN really shows – NB models predict almost instantly while plain KNN takes noticeably longer, tuning + tree-based search brings the tuned KNN’s time down a lot but it is still slower than any NB variant.

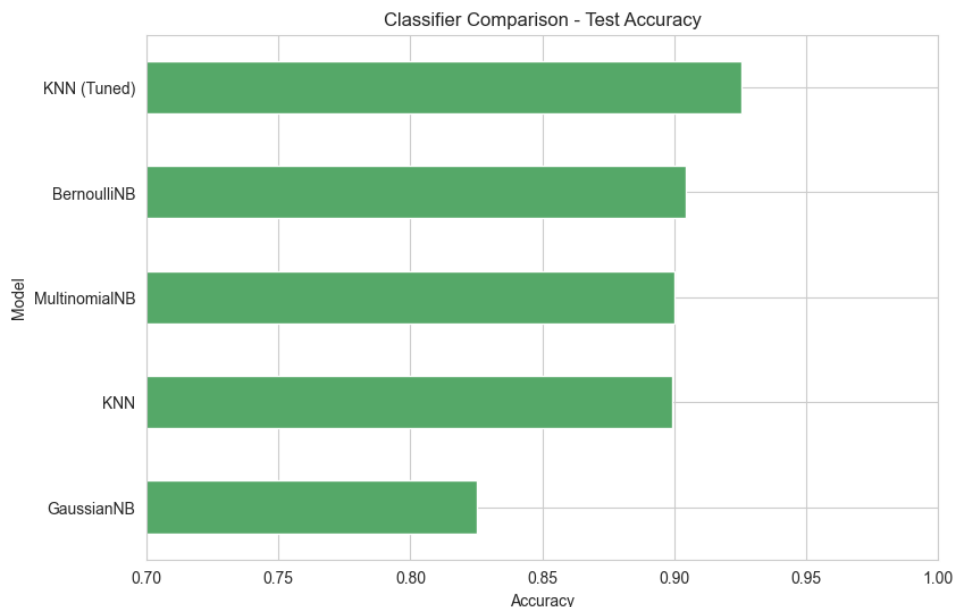


Figure 15: Overall classifier comparison – test accuracy

Inference: Tuned KNN comes out on top, followed by BernoulliNB, then MultinomialNB and plain KNN which are almost tied, with GaussianNB trailing behind all of them.

12 Analysis Questions

1. Which Naïve Bayes variant performed best?

BernoulliNB, with the highest accuracy (0.9044), F1 (0.8739) and ROC-AUC (0.9623) among the three variants, and it also had the best 5-fold CV mean (0.9022). Makes sense since a lot of the strongest features (like whether a spammy word appears at all) are naturally close to binary in behaviour.

2. What is the optimal value of k ?

$k=9$ gave the best test accuracy (0.9062) in the plain sweep. When distance metric and weighting were also tuned together via GridSearchCV, the overall best combo also landed on $k=9$ (with manhattan distance and distance weighting), so 9 seems to be a genuinely good choice here, not just a fluke of one hyperparameter.

3. Compare GridSearchCV and RandomizedSearchCV.

GridSearchCV got a marginally better CV accuracy (0.9226 vs 0.9220) since it checks every combination, but it took about 3.5x longer (9.71s vs 2.78s) to do so. For a small grid like this the difference barely matters, but for bigger search spaces RandomizedSearchCV would be the more practical choice since the accuracy gap it gives up is tiny.

4. Compare KDTree and BallTree.

Both gave identical accuracy (0.9062, expected since they are just different search strategies for the same neighbours). BallTree was faster on both training (0.0207s vs 0.0486s) and prediction (0.2406s vs 0.2661s) for this dataset, probably because Spambase has 57 dimensions which is on the higher side for KDTree to stay efficient.

5. Compare theoretical and practical complexity.

Theory says NB prediction is $O(d)$ (basically instant) and that matches what was measured, NB prediction times were all under 0.003s. KNN brute-force prediction is $O(nd)$

which should be the slowest, and it was, by far (0.576s). Tree structures are supposed to bring this down to roughly $O(\log n)$ on average and they did help, but the improvement was not massive here (0.27s vs 0.24s), likely because with 57 features the "curse of dimensionality" stops the tree from pruning the search space as effectively as it would in low dimensions – theory holds in the general shape of the result but the constant factors and dimensionality effects clearly matter a lot in practice.

6. Which classifier is preferred for large datasets?

Naïve Bayes, specifically BernoulliNB here. Its training and prediction complexity does not depend on comparing against every other data point like KNN does, so it will keep scaling well as the dataset grows, while KNN's prediction time (even with a tree) would keep climbing.

13 Additional Tasks

Different train-test splits: Only a single 75/25 stratified split was actually run in this notebook. Since KNN's 5-fold CV mean accuracy (0.9257) came out close to its single-split test accuracy (0.9253), the model does not look overly sensitive to the exact split chosen, though this was not separately tested with, say, a 60/40 or 80/20 split.

Euclidean vs Manhattan distance: This was covered as part of the GridSearchCV/RandomizedSearchCV metric option. Manhattan distance won in both searches, beating Euclidean and (in the randomized search) Chebyshev too. This kind of makes sense for this dataset since a lot of the features are frequency counts near zero, and Manhattan distance tends to be less sensitive to a few large outlier values than Euclidean.

Weighted KNN: Also covered via the `weights` parameter. `weights="distance"` (closer neighbours count more) beat `weights="uniform"` in both searches, and the GridSearchCV heatmap (Figure above) shows this pattern holding fairly consistently across most `k` values, not just at the optimum.

Overfitting (small `k`) vs Underfitting (large `k`): At `k=1` the model is very sensitive to individual noisy points (each prediction depends on exactly one neighbour), this is the classic overfitting regime, accuracy at `k=1` (0.8975) is already a bit lower than the peak at `k=9`. As `k` grows past 9 the decision boundary gets smoother and starts ignoring local structure, accuracy slowly drops off down to 0.8931 by `k=27`, this is underfitting. The accuracy-vs-`k` plot (Figure 9) shows this rise-then-fall pattern clearly, even if the swing itself is not very dramatic for this particular dataset.

14 Discussion

Overall the results make intuitive sense once you look at what each algorithm is actually doing. GaussianNB assumes normally-distributed features, but a lot of these word-frequency columns are heavily skewed (mostly zero, occasional spikes) which is probably why it underperforms compared to Bernoulli and Multinomial, its independence + normality assumptions are both being stretched here. BernoulliNB does the best among the NB variants, likely because presence/absence of certain words is a strong enough signal on its own, without needing exact frequency counts.

For KNN, tuning genuinely helped – going from the default `k=5` (0.8992 accuracy) to the properly tuned version (0.9253 accuracy, plus a good F1 bump too) is not a small jump. But this improvement comes at a real cost in interpretability and complexity, plus

prediction is still noticeably slower than any of the NB models even after tuning and using a tree structure.

If I had to actually deploy one of these for a live spam filter that has to handle a lot of emails per second, BernoulliNB looks like the more practical pick since it is only slightly behind the tuned KNN on accuracy (0.9044 vs 0.9253) but is roughly 50x faster at prediction time. Tuned KNN would make more sense if squeezing out every bit of accuracy really mattered and the prediction volume was low enough that the extra latency was not an issue.

One limitation worth calling out: the 391 duplicate rows were never removed, so there is a small chance the same email pattern shows up in both the train and test split, which could be inflating the reported numbers slightly. This should probably be fixed and the whole experiment re-run in a later version.

15 Conclusion

Four classifiers – GaussianNB, MultinomialNB, BernoulliNB and KNN – were trained and compared on the Spambase dataset for email spam detection. BernoulliNB came out as the strongest Naïve Bayes variant (Accuracy 0.9044, F1 0.8739), while KNN, after tuning k , the distance metric and the weighting scheme via GridSearchCV, ended up as the single best performing model overall (Accuracy 0.9253, F1 0.9009, ROC-AUC 0.9736). GridSearchCV and RandomizedSearchCV gave near-identical results, with RandomizedSearchCV being considerably faster, and BallTree slightly outperformed KDTree for this particular (57-dimensional) dataset. Cross validation confirmed these rankings were not just a one-off result of a lucky test split. In terms of practical deployment, Naïve Bayes still wins on raw speed, so the “best” model here really depends on whether accuracy or latency matters more for the use case.

16 References

- [1] Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed., O’Reilly Media, 2019.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [4] Kaggle, “Spambase Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>.
- [5] Scikit-learn Documentation – Naive Bayes, Nearest Neighbors, Model Selection. [Online]. Available: <https://scikit-learn.org/stable/>.